

SEO 的思考

1. Search Engine 与 Rag 流程对比:

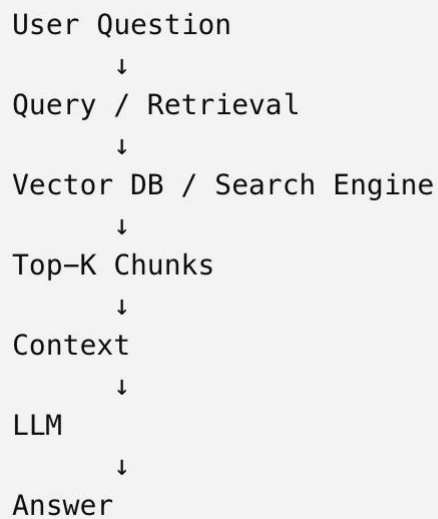
SE:



```
graph TD; A[用户 Query] --> B[Query Understanding]; B --> C[搜索索引 Index]; C --> D[召回候选网页]; D --> E[Ranking]; E --> F["Top 10 / Top N"]; F --> G[SERP];
```

用户 Query
↓
Query Understanding
↓
搜索索引 Index
↓
召回候选网页
↓
Ranking
↓
Top 10 / Top N
↓
SERP

RAG:



```
graph TD; A[User Question] --> B["Query / Retrieval"]; B --> C["Vector DB / Search Engine"]; C --> D["Top-K Chunks"]; D --> E[Context]; E --> F[LLM]; F --> G[Answer];
```

User Question
↓
Query / Retrieval
↓
Vector DB / Search Engine
↓
Top-K Chunks
↓
Context
↓
LLM
↓
Answer

2. 文档切割&检索

2.1 文档预处理

文档检索
表格检索
数据库检索

2.2 Chunking - 分片

字数分片
段落分片
语意分片

3. 倒排索引 (Inverted Index) : 哪些文档可能相关

3.1 正向索引

假设我们有 3 篇文章：

```
Doc1: I love apple  
Doc2: I love banana  
Doc3: Apple is delicious
```



如果按照我们平常的思维存储，就是：

```
Doc1 → I, love, apple  
Doc2 → I, love, banana  
Doc3 → apple, is, delicious
```



这叫比较直观的正向索引 (Forward Index)：

```
Document → Words
```



3.2 倒排索引

倒排索引直接建立：

Word → Documents

例如：

```
apple
  → Doc1
  → Doc3

love
  → Doc1
  → Doc2

banana
  → Doc2

delicious
  → Doc3
```



倒排列表：

```
apple →
[
  {
    docId: 1,
    positions: [3]
  },
  {
    docId: 3,
    positions: [1]
  }
]
```

也就是说：

某个词出现在哪些文档、出现了几次、出现在什么位置。

3.2.1 倒排索引建立的过程



举例：

第一步：文档进入 Indexer

搜索引擎首先拿到文档：

Document 1

Document 2

Document 3

...

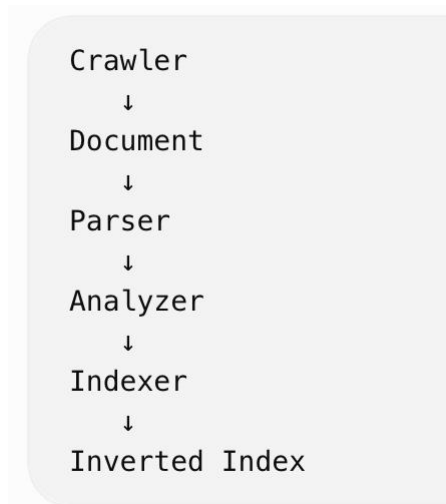
例如：

D1:

"Apple releases a new iPhone"

然后进入搜索引擎的 **Indexing Pipeline**。

典型架构：



第二步: Tokenization

把文本拆成一个个 Term。

例如:

"Apple releases a new iPhone"

可能得到:

Apple

releases

a

new

iPhone

但是搜索引擎通常不会直接使用原始字符串, 而会进行进一步的 Analysis。

例如:

"Running shoes are great"

经过分析可能变成:

run

shoe

great

第三步: 建立 Posting List

假设:

D1: "apple phone"

D2: "apple computer"

D3: "apple phone great"

分析之后:

D1 → apple, phone

D2 → apple, computer

D3 → apple, phone, great

Indexer 就开始建立:

apple
→ D1
→ D2
→ D3

phone
→ D1
→ D3

computer
→ D2

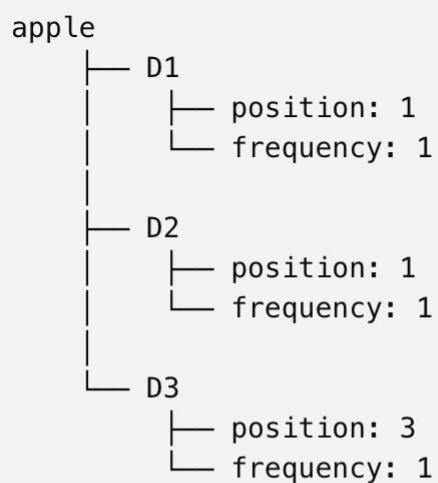
great
→ D3

其中:

apple → D1, D2, D3

D1, D2, D3 就叫 **Posting List**。

实际的 Posting 不只是 Document ID



因此 Posting 可能包含:

Document ID
Term Frequency
Position
Field
Offsets
...

为什么需要 Position?

判断两个词是否连续出现，连续出现可以判断这是一个 **Phrase Match**。

第四步: Term Dictionary

如果 Posting List 是:

apple → D1,D2,D3

phone → D1,D3

computer → D2

great → D3

搜索引擎还需要解决一个问题:

用户搜索 apple 时, 怎么快速找到 apple 这个 Term?

所以需要有一个 **Term Dictionary**。

可以理解成:

Term Dictionary

apple

computer

great

phone

...

通常还会进一步建立类似:

Term

↓

Posting List 的位置

例如:

apple → offset 1000

computer → offset 1500

great → offset 1800

phone → offset 2200

于是:

搜索 apple

↓

Term Dictionary

↓

找到 apple

↓

定位 Posting List



D1,D2,D3

3.3 搜索“apple”时发生了什么？

```
"apple"
  ↓
Tokenization
  ↓
apple
  ↓
Lookup Index
  ↓
apple → Posting List
  ↓
[Doc1, Doc3]
```

然后搜索引擎只需要对：

```
Doc1
Doc3
```

倒排索引解决的是：哪些文档可能相关？

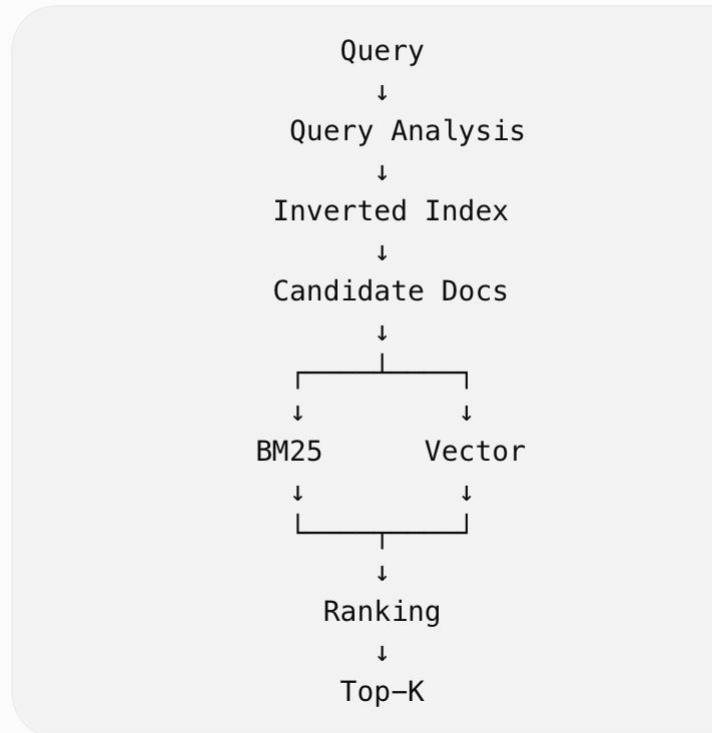
3.4 SEO 的启示

文档中应该多出现关键词更有利于索引

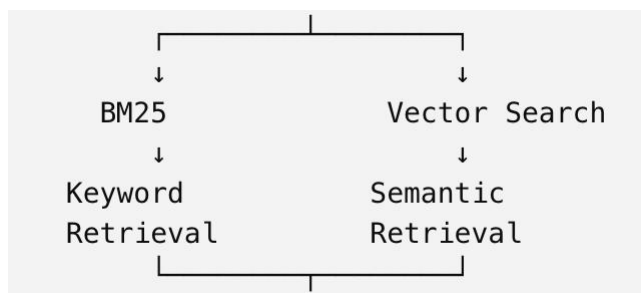
4. Ranking -- 哪些文档最相关

4.1 搜索系统

所以搜索系统通常是：



4.2 Hybrid Search (混合搜索)



4.3 BM25 相关性算法 (Best Matching 25) -- Keyword Retrieval

BM25 通常用于候选文档的快速相关性计算，而不是扫描所有文档。

BM25 会综合考虑：

- Query 词是否出现
- 出现次数
- 文档长度
- 这个词在整个语料库中有多常见
- TF / IDF 等因素

然后得到一个 **relevance score**。

BM25
=
IDF
×
TF Saturation
×
Length Normalization

也就是：

词的重要程度
×
词在文档中的重要程度
×
文档长度修正

4.4 BM25 和 GOOG 的关系

这里一定要区分：

Google $\neq$ 单纯使用 BM25 排名。

现代搜索引擎非常复杂。

Google会综合大量信号，例如：

```
Query relevance
Content relevance
Links
Page quality
Freshness
Entity
User intent
Location
Language
Page experience
Semantic understanding
...
```



BM25属于：

经典信息检索（Information Retrieval）方法。

4.5 BM25 对 SEO 的启示

文档的长度会对 **Keywords** 的相关度产生影响，词频相同的情况下，文档越长相关度越差。

4.6 Vector Search -- Semantic Retrieval

5. RRF（Reciprocal Rank Fusion）

当 BM25 和 Vector Search 各自返回：

BM25：

Doc A #1
Doc B #2
Doc C #3

以及：

Vector：

Doc C #1
Doc D #2
Doc A #3

问题是：

BM25的score和Vector similarity score不能直接比较。

RRF 不关心 Score,只关心排名。

5.1 RRF 算法

经典 RRF：

$$RRF(d) = \sum_{r \in R} \frac{1}{k + rank_r(d)}$$

其中：

- d : 某个 Document
- R : 所有 Retriever / Ranking List
- $rank_r(d)$: Document 在 Retriever r 中的排名
- k : 一个常数

最常见的：

$k = 60$

举例：

Chat

🔒 Work

Doc A

BM25:

Rank = 1

Vector:

Rank = 3

所以:

$$RRF(A) = \frac{1}{60 + 1} + \frac{1}{60 + 3}$$

大约:

$$\begin{aligned} &0.01639 + 0.01587 \\ &= 0.03226 \end{aligned}$$

Doc B

BM25:

Rank = 2

Vector:

Rank = 5

所以:

$$RRF(B) = \frac{1}{62} + \frac{1}{65}$$

大约:

0.01613 + 0.01538
= 0.03151



最终:

大概:

Doc A → 0.03226
Doc C → 0.03226
Doc B → 0.03151
Doc D → 0.01563
Doc E → 0.01538
Doc F → 0.01613
Doc G → 0.01563

排序后:

A / C
B
F
D / G
E

k 越小, 越强调排名靠前; k 越大, 排名之间的差异越平滑。

5.2 RRF 算法对 SEO 的启示

1. 一个文档只要要在多个 Retriever 中都表现不错，就很容易获得较高的最终排名。
2. Hub Page 的应用。
3. 两个检索系统都认为“还不错”的文档，通常比一个系统认为“极好”、另一个系统认为“很差”的文档更可靠。

假设：

Doc A:

BM25 #1

Vector #100

而：

Doc B:

BM25 #10

Vector #10

RRF:

$$A = 1/(61) + 1/(160)$$

$$B = 1/(70) + 1/(70)$$

大约：

$$A = 0.01639 + 0.00625 \\ = 0.02264$$

$$B = 0.01429 + 0.01429 \\ = 0.02858$$

所以：

$$B > A$$

4. 出现在多个检索器中的文档天然获得“共识奖励”。

Doc A

没有出现在 Vector Top 100。

通常可以理解为：

$$\text{Vector Rank} = \infty$$

也就是：

$$\text{Vector contribution} = 0$$

所以：

RRF(A)

=

BM25 contribution

6. Rerank

RRF负责“把多个检索器的结果合并起来”；Reranker负责“重新判断这些候选结果到底谁最相关”。

所以它们不是重复，而是前后两级筛选。

Reranker拿到：

Query
+
Candidate Documents

然后逐个判断：

这个 Document 和这个 Query 到底有多相关？

6.1 两阶段检索

经典架构：



第一阶段：查全率

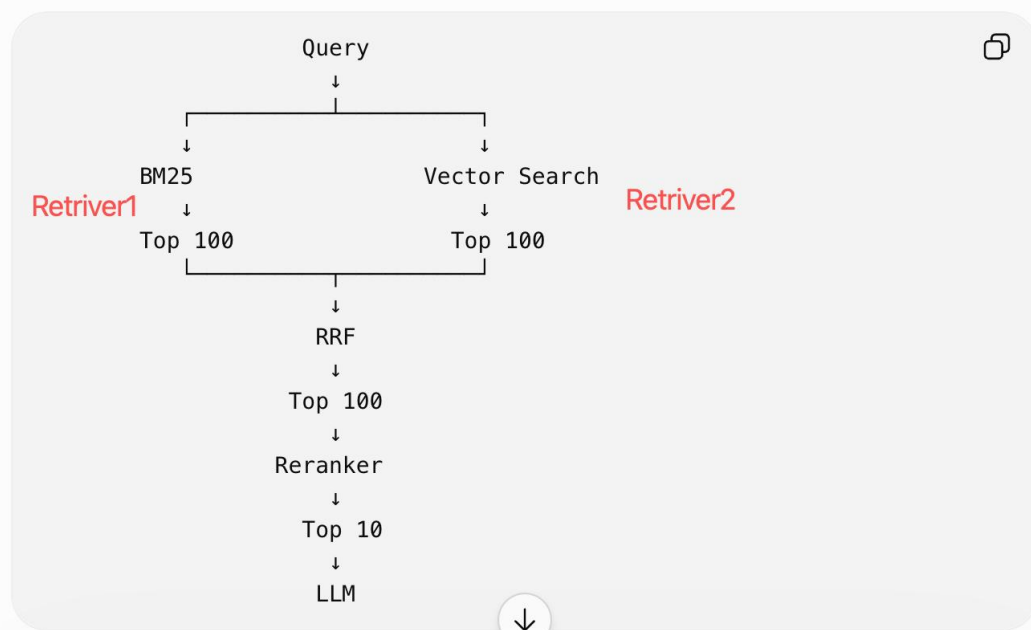
Recall / Candidate Retrieval

第二阶段：查准率

Precision / Reranking

6. 为什么RRF恰好适合放在中间?

因为你可能有多个 Retriever:



BM25:关键词召回

Vector:语义召回

RRF:多路结果融合

Reranker: 精确判断 Query-Document relevance

RRF 提高召回覆盖面, Reranker 提高最终准确率。

6.1 Cross-Encoder 算法

Cross-Encoder 不会分别计算 Query 和 Document 的向量，而是：

Query + Document
↓
Transformer
↓
Relevance Score



输入类似：

[CLS]
How can I humanize AI-generated text?
[SEP]
Humanizers can rewrite AI-generated content
to make it sound more natural...
[SEP]



然后模型输出：

0.94



Query 和 Document 被放进了同一个 Transformer 输入序列。

Cross-Encoder 本身不是一种全新的 Transformer 架构。

通常就是：

BERT / RoBERTa / DeBERTa 等 Transformer Encoder + 一个分类/回归 Head

最简单的 Cross-Encoder：

Transformer
↓
[CLS] embedding
↓
Linear Layer
↓
Score

Transformer Encoder 完成文本在不同维度的向量表示

分类/回归 Head 完成最后的建模，

分类模型输出：分类的值

回归模型输出：预测值

例如:

分类:

0 = Not Relevant

1 = Relevant

模型输出: [0.05, 0.95]

可以认为: $\text{relevance} = 0.95$

回归:

直接输出: 0.93

6.2 为什么很多 Reranker 模型是 BERT 系?

因为 Cross-Encoder 需要:

非常强的文本交互能力。

BERT 的双向 Self-Attention 很适合做:

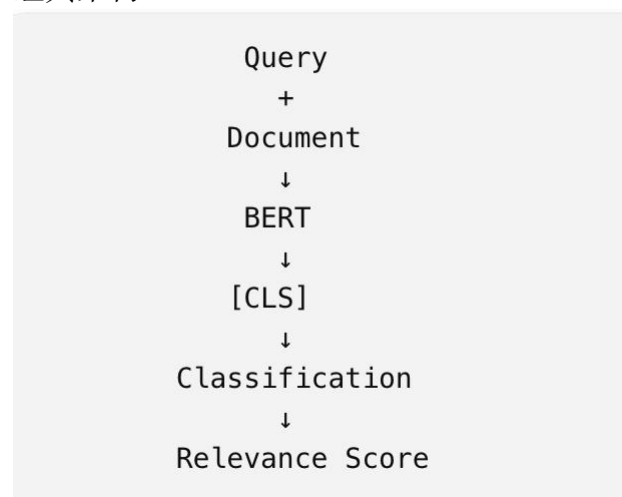
Query Token



Document Token

之间的交互。

经典架构:

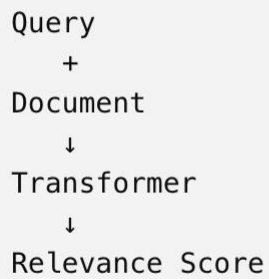


6.3 Bi-Encoder vs Cross-Encoder

Bi-Encoder



Cross-Encoder

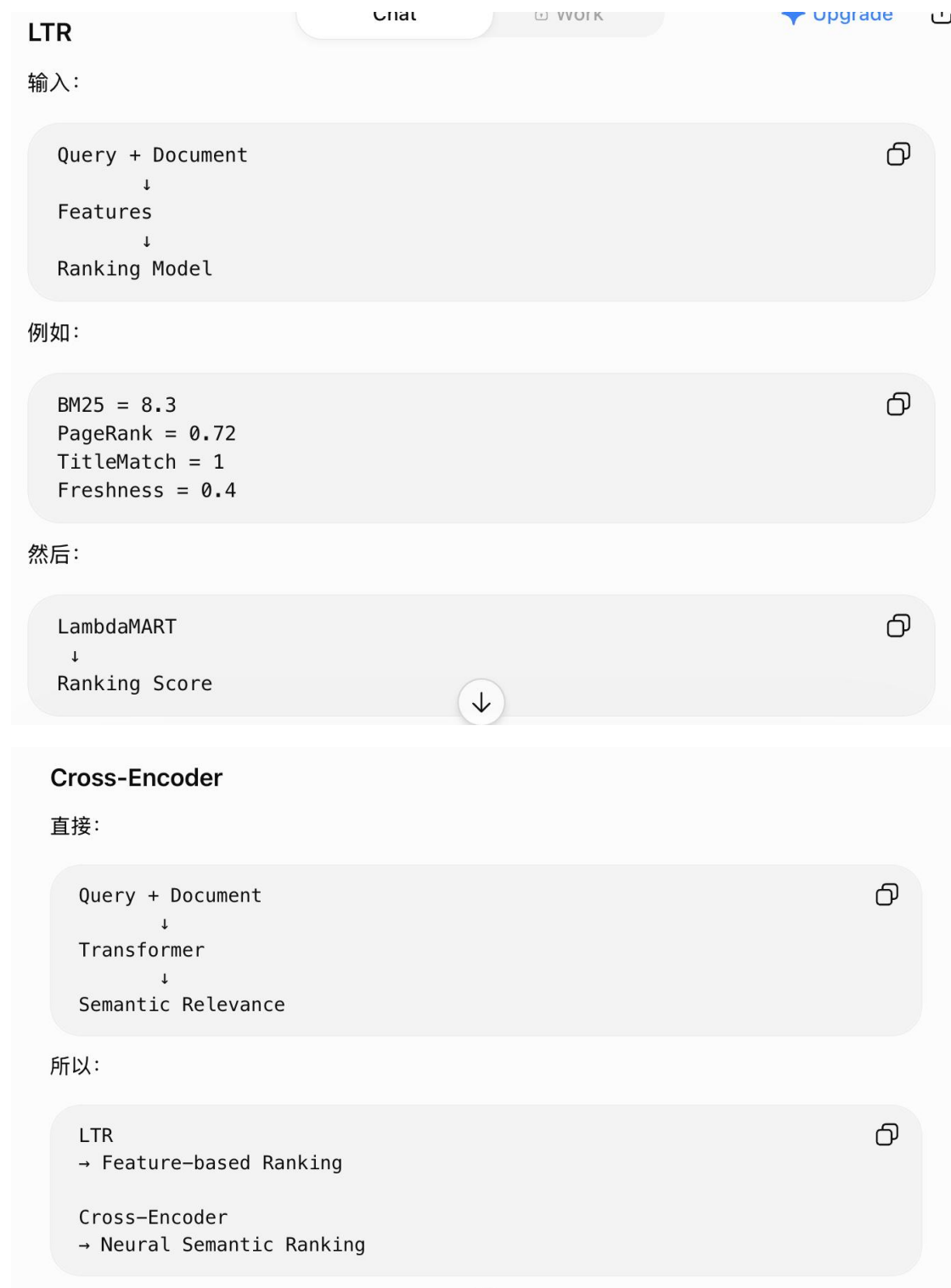


两个文本一起编码。

Bi-Encoder: 快，适合 Recall

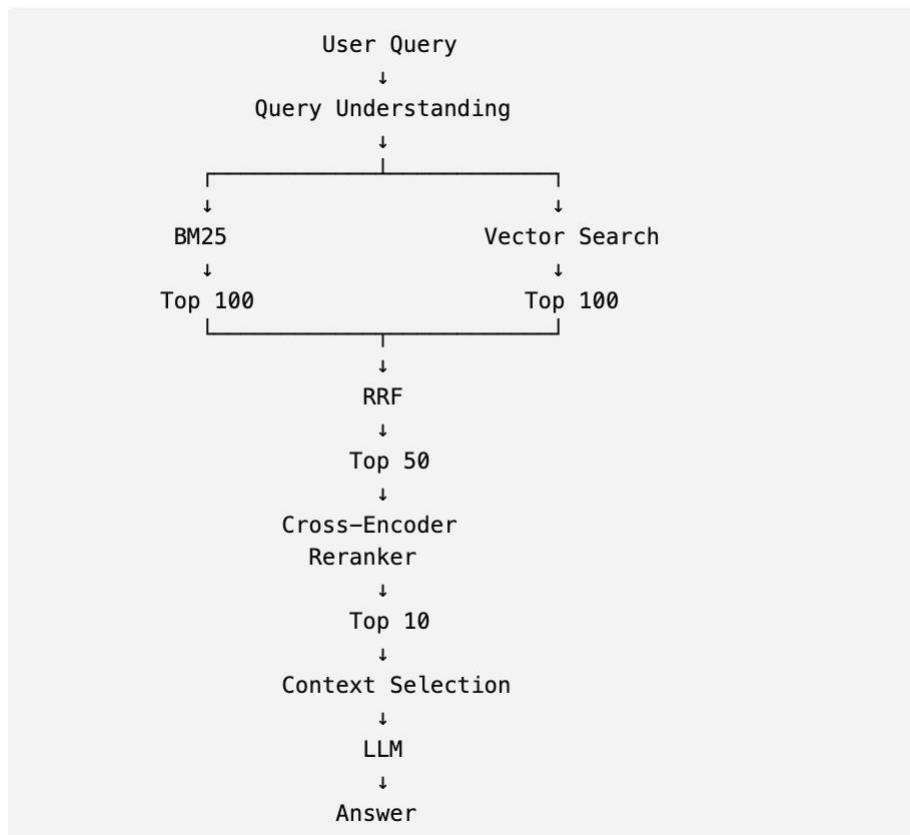
Cross-Encoder: 慢，适合 Reranking

6.4 Learning-to-Ranking vs Cross-Encoder

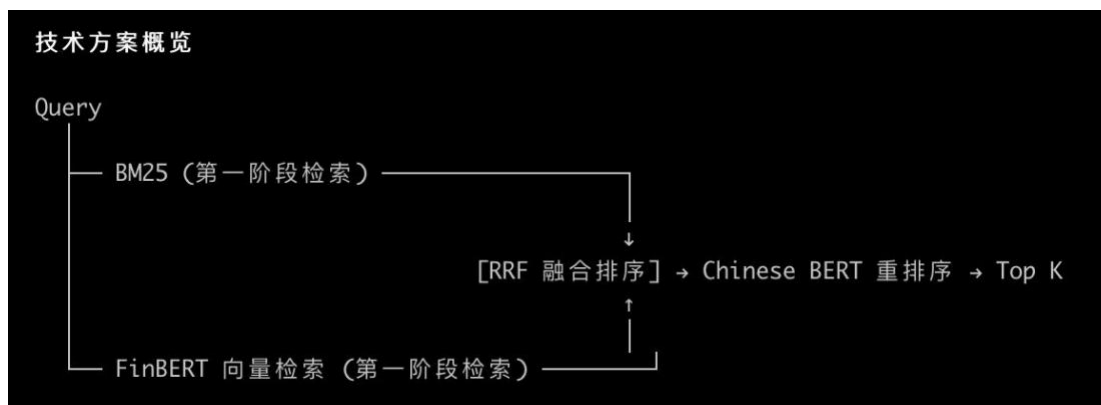


7.Rag

7.1 生产级的 Rag 架构



7.2 FinAgent Rag 架构



8.SEO

编入索引：网页性能 + sitemap

排名：Query 与文档的**相关度**决定排名

文档中应该多出现关键词更有利于索引

文档的长度会对 **Keywords** 的相关度产生影响，词频相同的情况下，文档越长相关度越差。

一个文档只要在多个 **Retriever** 中都表现不错，就很容易获得较高的最终排名。
两个检索系统都认为“还不错”的文档，通常比一个系统认为“极好”、另一个系统认为“很差”的文档更可靠。
出现在多个检索器中的文档天然获得“共识奖励”。
Hub Page 的应用。

9. QA

1. BM25 和 TF-IDF 是一个算法么？

不是。BM25 和 TF-IDF 不是同一个算法，但它们属于同一条技术路线，而且 BM25 可以看作是在 TF-IDF 思想基础上的进一步改进。

可以先记住：

TF-IDF：经典的词项相关性打分方法。

BM25：在 TF-IDF 基础上，对 TF 和文档长度做了更合理的处理。

TF-IDF=TF × IDF 的问题：

TF 增长得太简单。

出现1000次的文档，TF贡献可能是出现1次的1000倍。

显然不太合理。

例如一个网页：

```
humanizer humanizer humanizer humanizer...
```

不能因为疯狂重复关键词，就比正常文章相关1000倍。

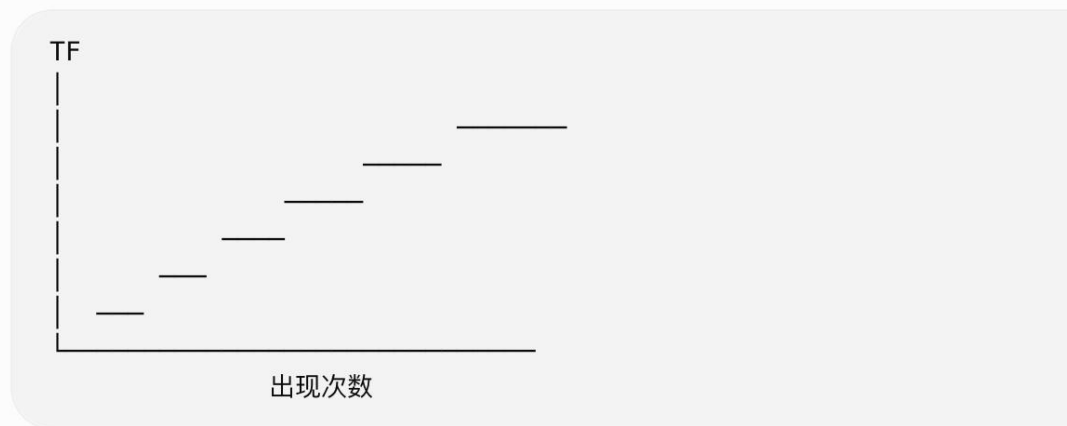
BM25 最大的改进之一: TF Saturation (词频饱和)

它不会让:

TF = 1
TF = 10
TF = 100

线性增长。

而是类似:

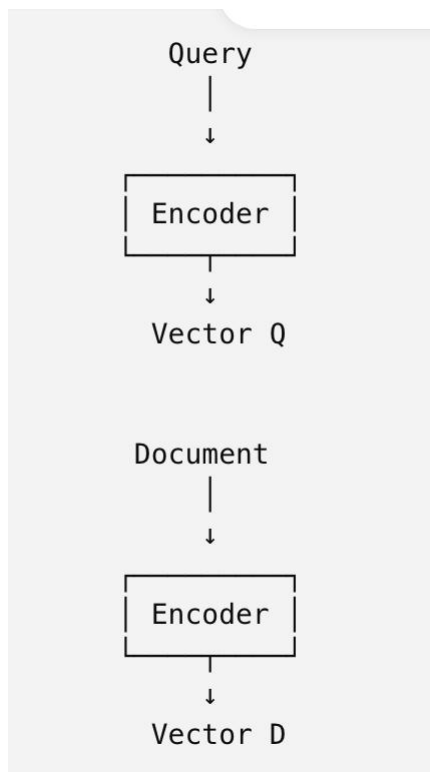


也就是:



前几次出现很重要, 后面继续重复的边际收益越来越小。

2. Vector Search 通常使用的算法是 Bi-Encoder 么?



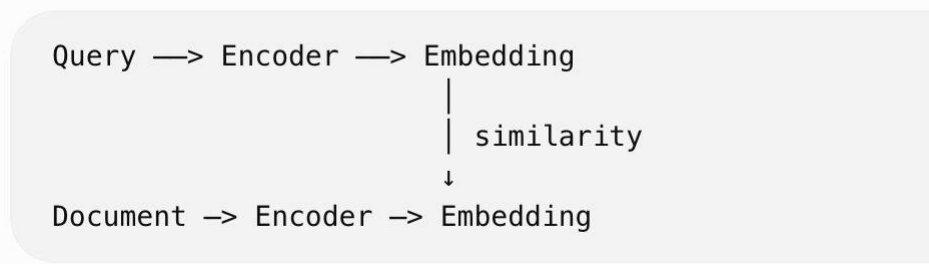
通常会即一步计算 Vector Q 和 Vector D 的相似度。

3. Bi-encoder 和 auto-encoder 的区别

Bi-Encoder 是“两个输入分别编码，用来比较两个对象是否相似”；
Auto-Encoder 是“把一个输入压缩，再从压缩结果中重建原输入”。

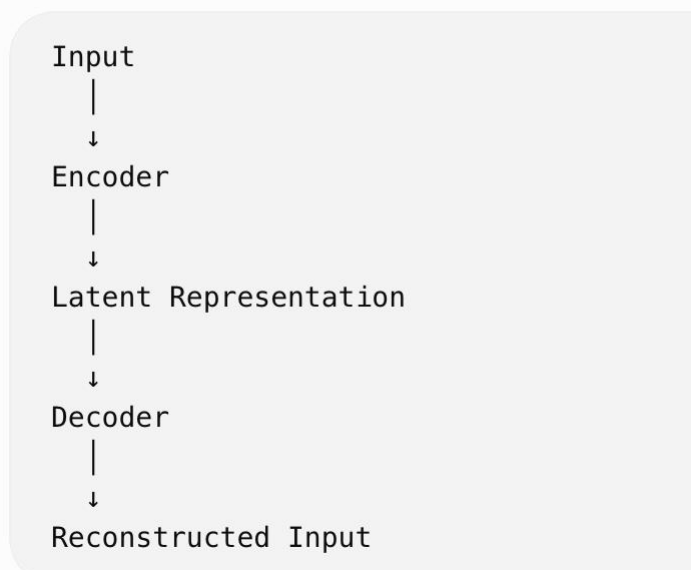
Bi-Encoder

典型结构：



Bi-Encoder 的核心是把不同对象映射到同一个向量空间，然后计算它们之间的关系。

Auto-Encoder 的结构是：



Auto-Encoder 的核心是：尽可能让 Decoder 根据压缩后的信息，把原始输入恢复出来。学习一个有信息压缩能力的 latent representation

	Bi-Encoder	Auto-Encoder	📄
核心目的	比较两个对象	压缩/重建一个对象	
输入	通常两个输入	一个输入	
Encoder	通常两个共享参数的 Encoder	Encoder + Decoder	
输出	embedding	latent representation + reconstruction	
是否需要 Decoder	✗	✓	
典型任务	Search、RAG、Retrieval、Similarity	降维、去噪、表示学习、重建	
常见训练目标	Contrastive Loss、Triplet Loss	Reconstruction Loss	
典型模型	SBERT	VAE、Denoising AE 等	

4.Tokenizer 和 Embedding 的区别

Tokenizer 负责把文字变成“模型认识的离散编号”；Embedding 负责把这些编号变成“模型可以计算的连续向量”。

Embedding 相当于把 Token 放在多维矩阵空间中表示。

假设 Vocabulary 有：

50,000 tokens

Embedding Layer 可以看成是一个巨大的矩阵：

Embedding Matrix

50000×768

也就是说：



5.判别式模型和生成式模型的区别

	BERT	GPT
类型	判别式倾向	生成式
主要任务	理解/表示	生成
训练目标	MLM	Next Token Prediction
输入	可以看双向上下文	通常只能看左侧上下文
擅长	分类、NER、Embedding	文本生成、对话
典型应用	文本分类	ChatGPT